

Project 3: Modular DBMS Engine

CMPE 321, Introduction to Database Systems, Spring 2026

1 Goal

In this project you will build a **modular DBMS engine** composed of four layers: a Disk Space Manager, a Buffer Manager, a File & Index Manager, and a Query Processor. Each layer is a Python module in its own folder. A single entry-point file (`archive.py`) wires them together using a JSON configuration file.

We will test your engine by running `archive.py` with **different configuration files** and identical inputs. By changing one line in the config (e.g., switching the replacement policy from LRU to MRU, or the index from `heap_scan` to `bplus_tree`) we observe how each design choice affects correctness and performance.

Every layer must communicate through **Result objects** — simple data containers that carry both the data and metadata between layers. Because all inter-layer communication flows through these objects, the modules remain true black boxes whose internals can be changed independently. **Design your Result objects carefully.**

2 File Structure

```
your_submission/
|-- archive.py                # Entry point: wires modules, runs input
|
|-- disk_space_manager/
|   |-- __init__.py          # exports DiskSpaceManager
|   |-- (your files)
|
|-- buffer_manager/
|   |-- __init__.py          # exports BufferManager
|   |-- (your files)
|
|-- file_index_manager/
|   |-- __init__.py          # exports FileIndexManager
|   |-- (your files)
|
|-- query_processor/
|   |-- __init__.py          # exports QueryProcessor
|   |-- (your files)
|
|-- workload_generator.py
|-- config.json
|-- README.md
|-- report.pdf
|-- ai_usage.md
|-- record.txt
|-- individual_contribution.pdf
```

Each module folder can contain as many internal files as you like. We only care about what each `__init__.py` exports. Your internal design is your own.

3 How archive.py Works

This is the **only file we run**. It imports the four module classes, reads the config, builds the layer stack, and processes the input. Below is the **exact pattern** your `archive.py` must follow. You may add helper code, but this structure must be preserved:

```
import sys, json

from disk_space_manager import DiskSpaceManager
from buffer_manager import BufferManager
from file_index_manager import FileIndexManager
from query_processor import QueryProcessor

def main():
    config_path = sys.argv[1]
    input_path = sys.argv[2]
    with open(config_path) as cf:
        config = json.load(cf)

    # Build layers bottom-up. Each layer receives the one below it.
    disk = DiskSpaceManager(config)
    buffer = BufferManager(config, disk)
    file_idx = FileIndexManager(config, buffer)
    qp = QueryProcessor(config, file_idx, buffer, disk)

    with open(input_path) as f:
        for line in f:
            line = line.strip()
            if line:
                qp.process(line)

    buffer.flush()

if __name__ == "__main__":
    main()
```

We will test your code exactly as: `python3 archive.py config.json input.txt` — we change only `config.json` between runs. Your code must produce correct `output.txt` for every valid configuration.

4 The Four Layers

Below we describe **what** each layer does and **what class name** it must export. We do not prescribe internal method names or signatures — that is your design. We treat each module as a black box. What matters is: (a) the class can be constructed as shown in Section 3, (b) the layers communicate through **Result objects** (see Section 5), and (c) the system produces correct output.

4.1 Layer 1 — DiskSpaceManager

Exported class: `DiskSpaceManager`

Constructor: `DiskSpaceManager(config: dict)`

The lowest layer. It is the **only** component that performs actual file I/O. It reads and writes

fixed-size pages to/from binary files on disk. It has no knowledge of records, indexes, or queries — only raw pages.

Responsibilities:

- Store each relation in a separate binary file composed of fixed-size pages.
- Read a single page from a file given a file identifier and page number.
- Write a single page to a file.
- Allocate new pages in a file.
- Track free space (free list or bitmap — your choice, document it).
- **Count every I/O operation** (read or write) and expose this count.
- Include a **log_write stub**: a callable that is invoked on every write. For now it can be a no-op, but it **must exist and be called on every write**.

Page size is set by the configuration file. Use Python's file I/O with `seek()` to read/write individual pages — never load an entire file into memory.

4.2 Layer 2 — BufferManager

Exported class: BufferManager

Constructor: BufferManager(config: dict, disk: DiskSpaceManager)

An in-memory page cache that sits between the File & Index Manager and the Disk Space Manager. When a page is needed, this layer checks its pool first. On a **hit**, no disk I/O occurs. On a **miss**, it evicts a page according to the active replacement policy, writes it back if dirty, and fetches the requested page from disk.

Responsibilities:

- Maintain a buffer pool whose size is set by the config.
- Implement at least two replacement policies: **LRU** and **MRU**.
- Track and expose: total requests, hits, misses, evictions, dirty writebacks.
- Mark pages as dirty when they are modified.

The active replacement policy is selected by the `replacement_policy` field in `config.json`. Layer 3 must **never** call DiskSpaceManager directly, it must always go through this layer.

4.3 Layer 3 — FileIndexManager

Exported class: FileIndexManager

Constructor: FileIndexManager(config: dict, buffer: BufferManager)

This layer understands records, relations (types), pages, and indexes. It uses the Buffer Manager for all page access.

Record & Page Organization

- Each type (relation) stored in its own file, consisting of multiple pages.
- Pages use the **unpacked slotted page format** with a bitmap indicating occupied slots.
- Records have **fixed length** within a type. Field definitions stored in a **System Catalog**.

- Up to 10 records per page. Only `int` and `str` field types. At least 6 fields per type.
- Type name max length: 12+ chars. Field name max length: 20+ chars.
- All type names, field names, and string field values consist of **alphanumeric characters only** (a–z, A–Z, 0–9). No spaces, special characters, or Unicode.
- When storing integer fields in pages, you must choose a fixed byte width (e.g., 4 bytes for signed 32-bit integers using Python’s `struct` module). When storing string fields, you must choose a fixed byte width and pad shorter values with null bytes. Document all sizing choices and their implications in your report.

Worked Example (one possible design): Suppose you choose `int` = 4 bytes (signed, via `struct.pack('i', val)`) and `str` = 32 bytes (null-padded). You design a page header of 64 bytes (page number, record count, 10-bit slot bitmap, padding). For a type with 3 `str` fields and 3 `int` fields:

`record_size` = $3 \times 32 + 3 \times 4 = 108$ bytes

`usable_space` = $4096 - 64 = 4032$ bytes

That fits $4032 // 108 = 37$ records, but we cap at 10 (from config). So $10 \text{ slots} \times 108 = 1080$ bytes used per page, with room to spare.

This is **one possible** design. You choose your own field widths, header layout, and record format. Different choices lead to different page capacities, B⁺-tree fanouts, and I/O characteristics, document and justify yours in the report or the beginning of the explanation video.

Index Strategies

Three access strategies, selected by config. Must be interchangeable without changing other layers:

- **heap_scan** — Sequential scan, no index. The baseline.
- **hash_index** — Static hash on primary key. Equality lookups only. Falls back to `heap_scan` for range queries.
- **bplus_tree** — B+ tree on primary key. Equality and range lookups.

Indexes must be built when a type is created and maintained on every insert/delete. Index data must be stored in pages that go through the Buffer Manager.

4.4 Layer 4 — QueryProcessor

Exported class: `QueryProcessor`

Constructor: `QueryProcessor(config: dict, file_idx: FileIndexManager, buffer: BufferManager, disk: DiskSpaceManager)`

The top layer. Parses input commands, delegates to `FileIndexManager`, collects statistics, handles the `explain` and `stats` commands, writes results to `output.txt`, and logs operations to `log.csv`.

The `QueryProcessor` receives references to all layers so it can query their statistics. It does not bypass layers for data access, it only reads counters from them.

5 Result Objects (Inter-Layer Communication)

This is the most important architectural requirement. Every time one layer calls another, the return value must be a **Result object**, a simple data container (e.g., a dataclass, named tuple, or plain class) that carries:

- **The data itself** (page bytes, record values, query results, etc.)
- **Metadata** about the operation (was it a cache hit? how many I/Os? how many pages scanned?)
- **A status** (success / failure / error message)

You decide the exact fields for each Result type. Below is an example to illustrate the pattern, you are free to use different names, add fields, or split into multiple Result types:

```
from dataclasses import dataclass
from typing import Optional, Any, List

@dataclass
class PageResult:
    """Returned by DiskSpaceManager and BufferManager when a page is fetched."""
    data: bytes                # the raw page bytes
    io_performed: bool         # True if a disk I/O actually happened
    # ... add whatever you need

@dataclass
class BufferResult:
    """Returned by BufferManager."""
    page: PageResult
    cache_hit: bool
    evicted_page_id: Optional[int] # None if no eviction
    dirty_writeback: bool         # True if evicted page was dirty

@dataclass
class RecordResult:
    """Returned by FileIndexManager for search/range operations."""
    records: List[Any]          # matching records
    pages_accessed: int
    index_nodes_visited: int    # 0 for heap_scan
    status: str                 # "success" or "failure"

@dataclass
class WriteResult:
    """Returned by DiskSpaceManager on write."""
    success: bool
    page_id: int
    old_data: bytes             # previous page content before this write
    new_data: bytes             # what was written
```

All inter-layer communication must flow through Result objects. If your layers pass raw values (plain bytes, bare integers) instead of Result objects, the modular design is broken. We must be able to swap one layer's output format without touching other layers.

6 Configuration File

```
{
  "page_size": 4096,
  "max_records_per_page": 10,
  "buffer_pool_size": 16,
  "replacement_policy": "LRU",
  "index_strategy": "bplus_tree"
}
```

Parameter	Values	Effect
page_size	4096 (default)	Bytes per page
max_records_per_page	up to 10	Slots per page
buffer_pool_size	4, 8, 16, 32, 64	Page frames in the buffer pool
replacement_policy	"LRU", "MRU"	Which page to evict when pool is full
index_strategy	"heap_scan", "hash_index", "bplus_tree"	How records are located

Invocation:

```
python3 archive.py config.json input.txt
```

7 Supported Operations

7.1 Data Definition

Operation	Format
Create Type	<code>create type <type-name> <num-fields> <primary-key-order> <field1-name> <field1-type> <field2-name> <field2-type> ...</code>

`primary-key-order` is 1-indexed. If it is 1, the first field is the primary key.

7.2 Data Manipulation

Operation	Format	Output (to output.txt)
Create Record	<code>create record <type> <v1> <v2> ...</code>	None
Delete Record	<code>delete record <type> <pk-value></code>	None
Search Record	<code>search record <type> <pk-value></code>	<code><v1> <v2> ...</code>
Range Search	<code>range_search <type> <field> <low> <high></code>	All matching records, one per line

Search/delete use the primary key. Range search works on any integer field. If `hash_index` is active and a `range_search` is requested, fall back to `heap_scan` for that query.

7.3 System Commands

Command	Format	Effect
Explain	<code>explain <any DML command></code>	Print plan before execution, then result, then actual stats (all to <code>output.txt</code>)
Stats	<code>stats</code>	Print all layer statistics to <code>stats_output.txt</code>
Stats Reset	<code>stats reset</code>	Reset all counters to zero

7.4 Failure Conditions

These must be logged as failure in `log.csv`. The system must not crash:

- Creating a type that already exists.
- Creating a record with a duplicate primary key.
- Deleting/searching a non-existing record or type.
- Range search on a non-integer field.

8 Log File

Every operation is logged to `log.csv` with: UNIX timestamp, the operation string, and **success** or **failure**. The log file is persistent, append-only, and must survive restarts. Use `int(time.time())` for timestamps.

```
1635018009,create type house 6 1 name str origin str leader str military_strength
    int wealth int spice_production int,success
1635018010,create record house Atreides Caladan Duke 8000 5000 150,success
1635018011,delete record house Corrino,failure
1635018012,search record house Atreides,success
```

9 Sample Input and Output

9.1 input.txt

```
create type house 6 1 name str origin str leader str military_strength int wealth
    int spice_production int
create record house Atreides Caladan Duke 8000 5000 150
create record house Harkonnen GiediPrime Baron 12000 3000 200
create record house Corrino Kaitain Emperor 15000 10000 50
search record house Atreides
delete record house Corrino
search record house Corrino
range_search house wealth 4000 9000
explain search record house Harkonnen
stats
```

9.2 output.txt

```

Atreides Caladan Duke 8000 5000 150
Atreides Caladan Duke 8000 5000 150
--- PLAN ---
Query:          search record house Harkonnen
Strategy:       bplus_tree
Estimated I/O:  3
--- RESULT ---
Harkonnen GiediPrime Baron 12000 3000 200
--- STATS ---
Actual I/O:     2 reads, 0 writes
Buffer Hits:    1
Buffer Misses:  1
Pages Scanned:  2

```

Note: `search record house Corrino` returns nothing (record was deleted), so nothing is printed to `output.txt` and it is logged as failure. The `range_search` returns Atreides (wealth=5000 is in [4000, 9000]).

9.3 stats_output.txt

When the `stats` command is executed, the following fixed format must be written to `stats_output.txt`. Each `stats` command **overwrites** the file (it reflects a snapshot, not a log):

```

=== STATISTICS ===
Disk I/O:      45 reads, 12 writes
Buffer Pool:   128 requests, 91 hits, 37 misses (71.1% hit rate)
Evictions:     29 (14 dirty writebacks)
Index:         bplus_tree, 23 nodes visited
Records:       82 scanned, 15 returned

```

All fields are mandatory. The `Index` line must reflect the active index strategy from the config. The `nodes visited` counter is cumulative since the last `stats reset` (0 for `heap_scan`).

10 Workload Generator

Include a script `workload_generator.py` that generates input files for your experiments. Keep it simple:

```

python3 workload_generator.py --mode MODE --records N --queries Q > workload.txt

Modes:
  sequential - Insert N records, then Q full-table scans
  random     - Insert N records, then Q random equality searches
  range      - Insert N records, then Q random range queries on an int field
  mixed      - Insert N records, then Q mixed ops (search / insert / delete)

```

The generator must create a type with at least 4 fields (including 2+ integer fields). It prints valid input lines to stdout.

11 Explanation Video

Your video must include an **explanation section** covering each implemented component. For every layer, describe its responsibilities, internal design, and how it interacts with adjacent layers.

11.1 Explanation 1 — Query Processor

Explain how the query processor parses and executes operations. Describe how it delegates work to lower layers and how results are written to `output.txt` and operations are logged to `log.csv`.

11.2 Explanation 2 — File & Index Manager

Explain your page structure and system catalog design in detail. Cover page layout (slots, bitmap, headers), record format, and how the catalog tracks type metadata. If indexing (Hash, B⁺-Tree) is implemented, describe its on-disk structure and lookup logic.

11.3 Explanation 3 — Buffer Manager

Explain your buffer pool implementation in detail. Cover how pages are pinned/unpinned, how dirty pages are flushed, and how both **LRU** and **MRU** replacement policies are implemented and toggled.

11.4 Explanation 4 — Disk Space Manager

Explain your I/O layer in detail. Cover how files are created, how pages are read/written individually, and how free space is tracked across pages and files.

12 Analysis Video

Your video must include a **comparative analysis section** with the following three experiments. For each experiment, describe your setup, present a results table, and explain *why* each strategy performs as it does.

12.1 Experiment 1 — LRU vs. MRU

Run the **sequential** and **random** workloads with both LRU and MRU using a fixed buffer pool size. Report I/O count and hit rate. Discuss the phenomenon of **sequential flooding** and which policy handles it better.

Table 1: LRU vs. MRU replacement policy comparison.

Workload	LRU I/Os	LRU Hit Rate	MRU I/Os	MRU Hit Rate
Sequential	...	...	...	...
Random	...	...	...	...

12.2 Experiment 2 — Index Strategy Comparison

Run equality and range queries using all three index strategies. Report I/O count for each.

Table 2: I/O comparison across index strategies.

Query Type	HeapScan	HashIndex	B ⁺ -Tree
Equality	...	...	...
Range	...	N/A (fallback)	...

12.3 Experiment 3 — Buffer Pool Size Sensitivity

Using a fixed workload and index strategy, vary the buffer pool size across **4, 8, 16, 32, and 64 pages**. Report I/O count and hit rate for each configuration.

Table 3: Effect of buffer pool size on I/O performance.

Buffer Size	I/Os	Hit Rate
4	...	...
8	...	...
16	...	...
32	...	...
64	...	...

Important

You must include a `record.txt` file containing the code and commands used for each experiment so that results can be independently reproduced.

13 Output File Locations

All output files are written to the **same directory as `archive.py`**. This includes:

- `output.txt` — query results and explain output
- `stats_output.txt` — statistics snapshots
- `log.csv` — persistent operation log
- All data files (relation files, index files, system catalog)

Do not write files to the current working directory, a temp folder, or any path derived from the config. Everything lives next to `archive.py`.

14 Technical Constraints

- Your implementation must use **only the Python standard library**. No third-party packages (no `pip install`). Standard modules such as `struct`, `os`, `json`, `time`, `sys`, `dataclasses` are all permitted.
- All input files use **ASCII-compatible encoding**. All type names, field names, and string values consist of alphanumeric characters only (a–z, A–Z, 0–9). No spaces, special characters, or Unicode.
- You choose your own field byte widths (int width, string width), page header layout, and record format. These choices affect page capacity, index fanout, and I/O characteristics. **Document and justify all sizing decisions in your report or the video.**

15 Persistence

All data files, the system catalog, index structures, and `log.csv` must be persistent across runs. If the engine is stopped and re-invoked with the same data directory, it must recover its state from disk.

16 Submission

For submission, submit one PDF file with the video links and a private GitHub repo link. We will try to reproduce the results.

17 Individual Contribution Report Guidelines

Each student must prepare an Individual Contribution Report and include it in the group ZIP file. The report must be in PDF format, named as: `Student ID_Contribution.pdf` (e.g., `123456_Contribution.pdf`).

What to Include:

- **Personal Information:** Name, Student ID, Group Number.
- **Tasks & Contributions:** Briefly describe what you worked on (schema normalization, triggers, procedures, backend, frontend, queries, documentation, debugging, etc.).
- **Collaboration & Challenges:** How the team worked together, any difficulties, and how they were resolved.

- **Self-Assessment:** Reflection on your role, skills learned, and areas for improvement.
- **Use of AI Tools:** Specify which AI tools were used (if any), how they were used for your specific tasks, and how they helped you obtain a better solution.

Formatting:

- **Length:** 1 page (max 2 pages)
- **Font & Size:** Times New Roman, 12pt, 1.5 spacing
- **File Format:** PDF

18 AI Usage

AI tools are permitted as learning aids. Include `ai_usage.md` describing what you used, what you asked, and what you changed. Honest disclosure is not penalized. You must be able to explain every part of your code.